C 语言伴读讲义 · 第 1 批

Ch1 初识C语言与程序框架

对应《C Primer Plus（第6版）》第 1–2 章合并 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：指针、位运算

1.1 学 C 之前：这语言为什么值得学

C 语言是很多大学电子信息/计算机专业的第一门编程课。它离"硬件"很近——你能直接控制内存地址、字节、硬件寄存器，这是 Python/Java 这些"更省事"的语言替你藏起来的东西。兰大的电子信息课程里，单片机的 C 代码、嵌入式、信号处理库，背后都是 C。

> **一句话定位：**C 让你看清"程序是怎么跑起来的"。哪怕以后主要用别的语言，C 建立的"内存、指针、编译"心智模型也会跟着你走。

但这门课有 **两座大山**：指针（第 10 章）和内存管理（第 12 章）。别怕——本讲义会在前面章节不断埋伏笔，到那两章你就不觉得陡了。

1.2 环境搭建：让"编译器"跑起来

你写的 C 代码（.c 文件）计算机不能直接运行，需要先用**编译器**把它翻译成机器能懂的机器码。本课程统一用 **MinGW-W64 的 gcc**（本机已装，16.1.0）。

三步走：

1. **写代码**：用编辑器（VS Code / Dev-C++ / 记事本都行）写一个 `hello.c` 文件，纯文本保存；
2. **编译**：命令行敲 `gcc -Wall -Wextra -o hello.exe hello.c`，生成可执行文件 `hello.exe`；
3. **运行**：敲 `.\hello.exe`，看到输出。

```
/* 编译 + 运行（在 hello.c 所在目录的终端里敲）*/
gcc -Wall -Wextra -o hello.exe hello.c   /* 生成 hello.exe */
.\hello.exe                              /* 运行，看到 "Hello, C!" */
```

> **为什么要 -Wall -Wextra?** 它让编译器把「可疑但还没到报错」的写法也提示出来（叫"警告"，warning）。初学者的 90% 低级错误——变量没初始化、类型不匹配、变量没用到——靠这两个选项提前暴露。养成习惯：写代码就开这两个选项，看到警告先修掉再运行，别"能跑就行"。

1.3 第一个程序：Hello, C!

每一个 C 程序都长这样（先背下来，后面所有章节都在这个骨架里加东西）：

```c
/* ch1_01_hello.c — 第一个程序 */
#include <stdio.h>          /* 引入"标准输入输出"库：printf/scanf 都在这里 */

int main(void)              /* 程序入口：程序从这里开始执行 */
{
    printf("Hello, C!\n");  /* 输出一行文字；\n 是换行 */
    return 0;               /* 告诉操作系统"正常结束" */
}
```

编译：`gcc -Wall -Wextra -o hello.exe hello.c`　运行：`.\hello.exe` → 输出：`Hello, C!`

四个部件逐个看：

- `#include <stdio.h>`：告诉编译器"我要用标准输入输出函数"。`stdio.h` 是"标准头文件"，里面声明了 `printf`/`scanf` 等。不 include 就用 `printf`，编译器会报"隐式声明"错误。
- `int main(void)`：每个程序必须有且只有一个 `main`，它是入口。`int` 表示 main 会返回一个整数，`void` 表示"不接受参数"。
- `printf(...)`：打印函数。`\n` 是"换行"转义字符。
- `return 0;`：返回 0 表示程序正常结束。返回非 0 通常表示出错。

1.4 编译错误三行对照：先学会读报错

零基础最先崩溃的不是写错逻辑，而是**看不懂编译器的报错**。养成习惯：看到报错，先看**行号**和**报错关键字**，再去那一行附近找问题。

报错原文（gcc 常见）	人话	怎么改
`error: expected '=', ',', ';', 'asm' or '__attribute__' before 'printf'`	少了分号	上一行末尾补 `;`（同类错误在不同上下文提示文字不同——此为本题实测原句）
`error: 'x' undeclared (first use...)`	变量 x 没声明	用之前先 `int x;` 声明
`warning: unused variable 'x'`	x 声明了没用	删掉，或真用起来
`error: implicit declaration of function 'printf'`	printf 没声明	检查 `#include <stdio.h>` 是否写上（**GCC 14 起为 error**，旧版仅 warning——本机 gcc 16.1.0 实测为 error）
`undefined reference to 'main'`	找不到 main	确认有 `int main(void)`

> **新手坑 1：报错行号只看个大概。**gcc 报的行号往往指向问题"所在的那一行附近"，真正的错误可能在前一行（比如漏了分号时，报错常落在下一行的 `}`）。从报错行往上多看两行，别死盯那一行。

1.5 数据和变量：程序怎么"记住"东西

程序要处理数据，就得先把数据放到"盒子"里——这个盒子叫**变量**。用之前必须先**声明**：告诉编译器"我要一个装整数的盒子，叫 num"。

```c
/* ch1_02_var.c — 声明、赋值、输出 */
#include <stdio.h>

int main(void)
{
    int age;          /* 声明：一个装 int（整数）的盒子，名叫 age */
    age = 19;         /* 赋值：往盒子里放 19 */

    int score = 95;   /* 声明 + 赋值，一步到位（初始化）*/

    printf("我 %d 岁，这次考了 %d 分\n", age, score);
    return 0;
}
```

运行输出：我 19 岁，这次考了 95 分

常用的基础类型先认识三个（更多在第 3 章 Ch3）：

类型	含义	示例
`int`	整数	`int n = 42;`
`double`	双精度小数	`double pi = 3.14159;`
`char`	单个字符	`char c = 'A';`

> **新手坑 2：变量没初始化就用。**声明了变量但没赋值（没放东西）就用，它的值是**内存里的残留值**——不确定、可能很大或很怪。这就是为什么 `-Wall -Wextra` 会警告 `uninitialized`。入门规矩：声明时就顺手给个初值（`int score = 95;`）。

1.6 新手坑集中营（本章最容易错的 3 件事）

1. **漏分号**：C 里每条"语句"（printf、赋值、return）结尾都要 `;`。漏了编译器从下一行才反应过来，报错看起来"错在别处"（见 1.4 对照表第一行）。
2. **变量没初始化就用**（见 1.5）——开 `-Wall` 让编译器替你抓。

3. **忘了 return 0**：C99 起 main 结尾缺 `return 0;` 编译器会补，但养成显式写的习惯，也避免有些环境告警。

1.7 一分钟自测（答案见练习册）

> **判断对错：**
> ① 程序必须先 `#include <stdio.h>` 才能用 `printf`。
> ② `int n;` 声明后，n 的初始值一定是 0。
> ③ `main` 是程序入口，每个程序只能有一个。

1.8 练习指引

做完本章内容后，去练习册完成 Ch1 配套练习；同时打开《C Primer Plus（第6版）》第 1、2 章，完成**复习题 1、2、3**（题号对应主教材）。改错题是练习册里最有价值的部分——先自己找 bug，再看答案的思路讲解。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译。原书题目请以正版教材为准。

C 语言伴读讲义 · 第 1 批

Ch3 数据和C

对应《C Primer Plus（第6版）》第 3 章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：位运算技巧、进制转换细节

3.1 数据在电脑里是怎么存的

计算机的"最小单位"是**位（bit）**，只能存 0 或 1。8 个位组成一个**字节（byte）**。一个 `int` 通常占 **4 字节 = 32 位**。你可以把变量理解成"一个有固定大小的盒子"——盒子多大（字节数），决定了它能装多大/多精的值。

> **一句话理解：**类型决定两件事——**占几个字节**（能存多大）和**怎么解释这些字节**（整数还是小数还是字符）。这就是为什么"类型不匹配"会出乱码。

3.2 整数类型：int 及其亲戚

类型	通常占用	范围（示例）
`short`	2 字节	约 -32768 ~ 32767
`int`	4 字节	约 -21亿 ~ 21亿
`long long`	8 字节	约 ±9.2×10^18

> **新手坑 1：int 溢出（最常见）。**盒子的容量是固定的，放进去的值超出范围就会"溢出"——结果像一个坏掉的里程表那样**绕回**。C 语言**不报错**，这是最危险的地方。

```c
/* ch3_01_overflow.c — int 溢出现象 */
#include <stdio.h>

int main(void)
{
    int big = 2147483647;   /* int 能装的最大值 */
    printf("big = %d\n", big);
    printf("big + 1 = %d\n", big + 1);   /* 溢出！会绕回成负数 */
    return 0;
}
```

运行输出（本机 gcc 16.1.0 实测）：`big = 2147483647  big + 1 = -2147483648`

为什么是 -2147483648？因为 int 用最高位表示正负（补码）。最大值再加 1，二进制全 1 进位翻到符号位，就成了最小的负数。记住现象即可：**超出范围会绕回，编译器不提醒，只能靠判断范围**。

3.3 浮点类型：float 和 double

存小数用浮点：float（4 字节，精度约 7 位）和 double（8 字节，精度约 15 位）。**日常运算默认用 double**，除非明确要求省内存。

```
/* ch3_02_float.c — 浮点精度 */
#include <stdio.h>

int main(void)
{
    float  f = 0.1f;
    double d = 0.1;
    printf("float 0.1  → %.10f\n", f);
    printf("double 0.1 → %.10f\n", d);
    return 0;
}
```

运行输出（实测）：`float 0.1 → 0.1000000015  double 0.1 → 0.1000000000`

新手坑 2：小数不"精确"。0.1 在二进制里是无限循环小数，存不下。float 会把误差放大——看到 0.1000000015 别惊讶。**结论：不要用 == 判断两个浮点数相等**（见练习册）；金额、分数这类需要精确的，要么用整数（分），要么用高精度库。

3.4 字符类型：char 和 ASCII

char 存**一个字符**，其实是存这个字符的 **ASCII 编码数字**。'A' 在内存里就是数字 65。

```
/* ch3_03_char.c — char 就是数字 */
#include <stdio.h>

int main(void)
{
    char c = 'A';
    printf("字符：%c\n", c);
    printf("它的 ASCII 数字：%d\n", c);   /* %c 当字符打，%d 当数字打 */
    return 0;
}
```

运行输出（实测）：`字符：A  它的 ASCII 数字：65`

> **伏笔：**因为字符就是数字，所以 `c + 1` 会得到 66（即 'B'）。这在后面做字母大小写转换时会用到。

3.5 常量：不打算改的值

写死在代码里的字面量（42、3.14、'A'）是**字面常量**。想给常量起个名字、防止手滑改掉，用 `const`：

```
/* ch3_04_const.c — const 常量 */
#include <stdio.h>

int main(void)
{
    const double PI = 3.14159;
    int radius = 10;
    printf("圆面积 = %f\n", PI * radius * radius);
    return 0;
}
```

运行输出（实测）：`圆面积 = 314.159000`

> **新手坑 3：**`PI = 3.0;` 这种对 `const` 变量的赋值，编译器会报错——这正是 const 想保护你的。别把 const 常量当变量改。

3.6 printf/scanf：%占位符要对上类型

上一章学过 printf/scanf，这一章重点是**类型必须对号入座**：

占位符	配什么类型	说明
`%d`	`int`	整数
`%ld`	`long`	长整数
`%f`	`float`（printf 里也接受 double）	小数，6 位
`%lf`	`double`	**scanf 读 double 必须 %lf**
`%c`	`char`	字符
`%s`	`char[]`	字符串

```
/* ch3_05_scanf.c — scanf 读 double 的坑 */
#include <stdio.h>
```

```c
int main(void)
{
    double height;
    printf("输入身高：");
    scanf("%lf", &height);   /* double 必须 %lf，且要 & */
    printf("身高 = %f\n", height);
    return 0;
}
```

> **新手坑 4：scanf 用 %f 读 double。**这是 Ch4 已埋的坑，再强调一次：printf 里 %f 能打 double（自动转），但 scanf 里**必须** %lf 配 double、%f 配 float——写反了读出来的值是乱码。

3.7 新手坑集中营（本章最容易错的 4 件事）

1. **int 溢出不报错**（见 3.2）——算大数前先估算范围。
2. **浮点数 == 判断相等**（见 3.3）——用 fabs(a-b) < 1e-6 这类容差判断。
3. **scanf 占位符和类型对不上**（见 3.6）——%d/%f/%lf 记牢。
4. **给 %d 传 double**：printf 里 printf("%d", 3.14) 输出是乱码（类型和格式对不上，gcc 开 -Wall 会警告）。

3.8 一分钟自测（答案见练习册）

判断对错：

① int n = 2147483647; n + 1 的结果一定是正数。

② scanf("%lf", &x) 可以读 double 类型。

③ 两个 double 用 == 判断相等是可靠的。

3.9 练习指引

做完本章内容后，去练习册完成 Ch3 配套练习；同时打开《C Primer Plus（第6版）》第 3 章，完成**复习题 1、3、5**（题号对应主教材）。改错题是练习册里最有价值的部分——先自己找 bug，再看答案的思路讲解。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 gcc -Wall -Wextra 编译，讲义每句「输出:」均已实跑核对。原书题目请以正版教材为准。

C 语言伴读讲义 · 样品章

Ch4 字符串和格式化输入/输出

对应《C Primer Plus（第6版）》第 4 章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：位运算、指针高级用法

4.1 先把程序框架摆正

每一个 C 程序都长这样（先背下来，后面所有章节都在这个骨架里加东西）：

```
/* ch4_01_framework.c — 程序基本框架 */
#include <stdio.h>          /* 引入输入输出函数（printf/scanf 都在这里）*/

int main(void)              /* 程序入口：从这里开始执行 */
{
    printf("Hello, C!\n");  /* 输出一行文字 */
    return 0;               /* 告诉系统"正常结束" */
}
```

编译：gcc -Wall -Wextra -o hello.exe hello.c 运行：.\hello.exe

> **为什么要 -Wall -Wextra?** 它让编译器把「可疑但没报错」的写法也提示出来。初学者的 90% 低级错误（变量没初始化、类型不匹配）靠它提前暴露。养成习惯：写代码就开这两个选项，看到警告先修掉再运行。

4.2 printf：把东西"打印"到屏幕上

printf 的完整格式是 `printf("格式字符串", 要输出的值们)`。格式字符串里用 **%占位符** 表示「这里放一个值」，后面的值按顺序对号入座：

占位符	含义	示例输出
%d	整数（int）	printf("%d", 42); → 42
%f	小数（float/double，默认 6 位小数）	printf("%f", 3.14); → 3.140000
%.2f	小数，保留 2 位	printf("%.2f", 3.14159); → 3.14
%c	单个字符	printf("%c", 'A'); → A
%s	字符串	printf("%s", "hi"); → hi

还可以控制宽度：`%5d` 表示「至少占 5 个字符位，右对齐」；`%-5d` 左对齐。转义字符 `\n`（换行）、`\t`（制表）也在这里用。

```
/* ch4_02_printf.c — 格式化输出练习 */
#include <stdio.h>

int main(void)
{
    int    age  = 19;
    double temp = 36.5;

    printf("我 %d 岁\n", age);
    printf("体温 %.1f 度\n", temp);
    printf("%-8s|%8s\n", "left", "right");   /* %-8s 左对齐占8位（ASCII 演示）*/
    return 0;
}
```

> **新手坑 4：中文对齐不按「字数」算。**`%10s` 的宽度单位是**字节**，中文一个字 3 字节（UTF-8），"左对齐"3 个字已占 9 字节 → 想对齐得放大宽度，或用 ASCII 演示。上面示例特意用 "left"/"right" 演示对齐——换中文试试，竖线就对不齐了。

4.3 scanf：让程序"读"你输入的东西

scanf 和 printf 长得像，但有一个**最容易翻车**的差别：给变量传值时，变量名前面要加 **&**（取地址符号）。

```
/* ch4_03_scanf.c — 读取键盘输入 */
#include <stdio.h>

int main(void)
{
    int num;
    printf("请输入一个整数：");
    scanf("%d", &num);          /* 注意 & ！没有它程序会崩或乱读 */
    printf("你输入的是 %d\n", num);
    return 0;
}
```

> **新手坑 1：忘了 &。**scanf 需要知道「把值存到哪个内存地址」，`%d` 配 `&num`。忘了写，轻则警告、重则段错误（程序崩溃）。
> **新手坑 2：%s 读不进空格。**`scanf("%s", name)` 遇到空格就停，输入 "Li Ming" 只会读到 "Li"。要读一整行，用 `fgets(name, sizeof(name), stdin)`（第 11 章细讲）。
> **新手坑 3：缓冲区残留换行。**连续 scanf 时，上一次按回车留下的 `\n` 可能被下一次 scanf 直接消费，表

现为「没让我输入就跳过了」。入门阶段先记住：`%c` 前面加个空格（`scanf(" %c", &ch)`）能吞掉残留换行。

4.4 字符串：其实就是字符数组

C 语言没有独立的「字符串类型」。字符串就是一个**以 \0（空字符）结尾的 char 数组**——最后一个元素一定是 \0，用来标记「字符串到这里结束」。

```
/* ch4_04_string.c — 字符串的本质 */
#include <stdio.h>

int main(void)
{
    char name[20] = "F-VIS";   /* 数组长度20，实际用了6个：F-V-I-S-\0 */
    printf("名称：%s\n", name);
    printf("第1个字符：%c\n", name[0]);   /* 数组下标从 0 开始 */
    return 0;
}
```

一句话理解内存：数组名 `name` 本身是「数组起始位置的地址」。`name[0]` 是第一个元素、`name[1]` 是第二个……这段「地址 + 下标」的心智模型，是第 10 章指针的地基——现在先种下。

4.5 新手坑集中营（本章最容易错的 4 件事）

1. **%d 配 double / %f 配 int**：格式字符串和值类型对不上，输出是乱码。printf 里 float 会自动转 double，但 scanf 不会——`%f` 读 float，`%lf` 读 double。
2. **scanf 忘记 &**（见 4.3）。
3. **字符串数组不够大**：`char s[4] = "hello"`——GCC 直接报警告 `initializer-string too long`，而且数组里没有 \0 的位置，`%s` 打印会越界读内存。开 `-Wall -Wextra` 让编译器提前拦住它。
4. **\n 忘写**：输出全部挤在一行，看不出结构。

4.6 一分钟自测（答案见练习册）

判断对错：

① `printf("%d", 3.14);` 是合法的，会正常输出 3。

② `scanf("%d", num);` 是正确的读入写法。

③ 字符串 "OK" 实际占 3 个 char 的位置（含 \0）。

4.7 练习指引

做完本章内容后，去练习册完成 Ch4 配套练习；同时打开《C Primer Plus（第6版）》第 4 章，完成**复习题 1、3、5**（题号对应主教材）。改错题是练习册里最有价值的部分——先自己找 bug，再看答案的思路讲解。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译。原书题目请以正版教材为准。

C 语言伴读讲义 · 第 2 批

Ch5 运算符、表达式与语句

对应《C Primer Plus（第6版）》第 5 章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：位运算、三目运算符

5.1 运算符：让程序"会算数"

上一章我们让程序"记住"了数据（变量），这一章让程序**算起来**。运算符就是告诉计算机"怎么算"的符号。最常用的五个：

运算符	含义	示例	结果
+	加	10 + 3	13
-	减	10 - 3	7
*	乘	10 * 3	30
/	除	10 / 3	3（int 除法，见 5.2）
%	求模（取余数）	10 % 3	1

```c
/* ch5_01_basic.c — 加减乘除（int）*/
#include <stdio.h>

int main(void)
{
    int a = 10, b = 3;
    printf("%d + %d = %d\n", a, b, a + b);
    printf("%d - %d = %d\n", a, b, a - b);
    printf("%d * %d = %d\n", a, b, a * b);
    printf("%d / %d = %d\n", a, b, a / b);
    return 0;
}
```

编译：`gcc -Wall -Wextra -o ch5_01_basic.exe ch5_01_basic.c`　运行：`.\ch5_01_basic.exe`

运行输出（实测）：`10 + 3 = 13  10 - 3 = 7  10 * 3 = 30  10 / 3 = 3`

5.2 整数除法会"丢尾巴"：/ 和 % 是一对

两个 int 相除，结果**还是 int**——小数部分直接丢掉（不是四舍五入，是**截断**）。10 / 3 = 3，剩下的那 1 就是余数，用 % 取出来。

> **口诀：** / 取商，% 取余。判断"能不能整除"就看 n % m == 0。想保留小数？让**其中一个数是 double**（见 5.7）。

```
/* ch5_02_divmod.c — 整数除法 + 求模 */
#include <stdio.h>

int main(void)
{
    int minutes = 125;
    printf("125 分钟 = %d 小时 %d 分钟\n", minutes / 60, minutes % 60);

    int n = 17;
    if (n % 2 == 0) printf("%d 是偶数\n", n);
    else            printf("%d 是奇数\n", n);
    return 0;
}
```

运行输出（实测）：125 分钟 = 2 小时 5 分钟 17 是奇数

5.3 自增 ++ 自减 --：写代码偷懒的常用手段

n++ 等价于 n = n + 1，n-- 等价于 n = n - 1。它们还能"边用边变"：

写法	含义	例子（n 初值 5）
n++（后缀）	先取 n 的旧值用，用完再加 1	x = n++; → x=5，n=6
++n（前缀）	先加 1，再取新值用	x = ++n; → n=6，x=6

> **新手坑：一个语句里别对同一个变量又 ++ 又用。**printf("%d %d\n", n, n++); 这种"边打印边改"的结果在 C 里**不确定**（不同编译器结果可能不同）。规矩：++/-- 单独成行用，别混进别的表达式里。

```
/* ch5_03_incdec.c — 前缀/后缀的区别 */
#include <stdio.h>

int main(void)
{
    int a = 5;
    int x = a++;      /* x 先拿 a 的旧值 5，然后 a 变成 6 */
    int y = ++a;      /* a 先变成 7，然后 y 拿新值 7 */
    printf("a=%d x=%d y=%d\n", a, x, y);
```

```
    return 0;
}
```

运行输出（实测）：a=7 x=5 y=7

5.4 赋值 = 与复合赋值：+= -= *= /=

= 是把右边的值"放进"左边的盒子（变量）。注意：**左边必须是变量**，不能是 a + b 这种表达式。

想"把变量自己加 5"，可以写 score = score + 5;，更短的写法是 score += 5;——这叫**复合赋值**：

```
/* ch5_04_compound.c — 复合赋值 */
#include <stdio.h>

int main(void)
{
    int score = 80;
    score += 10;    /* score = 90 */
    score -= 5;     /* score = 85 */
    score *= 2;     /* score = 170 */
    printf("score = %d\n", score);
    return 0;
}
```

运行输出（实测）：score = 170

5.5 优先级：先乘除后加减，别靠猜

数学里"先乘除后加减"，C 也一样。拿不准的时候**加括号**，括号优先级最高。

```
/* ch5_05_preced.c — 优先级与括号 */
#include <stdio.h>

int main(void)
{
    int r = 2 + 3 * 4;       /* 先乘：3*4=12，再加 2 → 14 */
    int s = (2 + 3) * 4;     /* 括号先算：5*4 → 20 */
    printf("r=%d s=%d\n", r, s);
    return 0;
}
```

运行输出（实测）：r=14 s=20

优先级（高→低，本章够用）	说明
( )	括号最高

`++ --`	自增自减
`* / %`	乘、除、求模（同级，从左往右）
`+ -`	加、减（同级，从左往右）
`= += -= ...`	赋值（最低）

> **一句话：**只要表达式里混着 >2 种运算符，就多用括号表达你的意图——既防错，别人（和考试判卷）也看得懂。

5.6 表达式与语句：分号是关键

表达式是"能算出值的一串东西"：`2 + 3`、`a * b`、`n++` 都是表达式。**语句**是"完整的一句话"，结尾必须带 `;`。最常见的语句就是"表达式 + 分号"：

```
/* 表达式 vs 语句 */
2 + 3;          /* 合法但没用的语句：算完就扔了 */
a = b + c;      /* 赋值语句（也是表达式加分号）*/
printf("hi");   /* 函数调用语句 */
```

> **新手坑：多余的分号。**在 `if` 后面多加一个分号会改变程序逻辑（第 6 章循环会再遇到）。判断"这里该不该有分号"：这句话是不是一个完整动作？是——加分号。

5.7 类型转换：int 和 double 混着算

整数和小数混在一起算，C 会把 `int` 自动"升级"成 `double` 再算（小的往大的转，不丢精度）。反过来 `double` 想存进 `int`，小数部分会被**截断**（这是警告级别的问题，开 `-Wall` 会提示）。

```
/* ch5_06_type.c — int/double 混合运算（练习册 ③ 有对应题）*/
#include <stdio.h>

int main(void)
{
    int a = 7;
    double half = a / 2.0;      /* a 是 int，2.0 是 double → 结果是 3.5 */
    int trunc = a / 2;          /* 两个 int → 3，小数被截断 */
    printf("half=%.1f trunc=%d\n", half, trunc);
    return 0;
}
```

运行输出（实测）：`half=3.5 trunc=3`

5.8 新手坑集中营（本章最容易错的 4 件事）

1. **整数除法丢小数**：`5 / 2 = 2` 不是 2.5。想保留小数：让任一操作数是 double（`5 / 2.0`）。
2. **自增自减混进表达式**：`printf("%d", n++)` 结果不确定。单独成行用。
3. **赋值方向写反**：`=` 是把右边给左边。`5 = a;` 编译就报错（左边必须是变量）。
4. **拿不准优先级不加括号**：混着算的表达式都加括号，别赌记忆。

5.9 一分钟自测（答案见练习册）

写出结果：

① `int n = 10; printf("%d", n / 3);`

② `int m = 5; int r = m++; printf("%d %d", m, r);`

③ `double x = 7; printf("%.2f", x / 2);`

5.10 练习指引

做完本章内容后，去练习册完成 Ch5 配套练习；同时打开《C Primer Plus（第6版）》第 5 章，完成**复习题 1、2、4**（题号对应主教材）。改错题是最有价值的——先自己找，再看答案。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译，每句「运行输出（实测）」均已实跑核对。原书题目请以正版教材为准。

C 语言伴读讲义 · 第 2 批

Ch6 C控制语句：循环

对应《C Primer Plus（第6版）》第 6 章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：goto、逗号表达式

6.1 while 循环：条件成立就一直做

程序最强大的能力是**重复**。让计算机打印 1 到 10，不需要写十行 printf——用循环。最基础的是 while：

```
/* ch6_01_while.c — while 数数 */
#include <stdio.h>

int main(void)
{
    int n = 1;
    while (n <= 5) {        /* 条件为真就进循环体 */
        printf("%d ", n);
        n++;                /* 计数器 +1，否则死循环 */
    }
    printf("\n");
    return 0;
}
```

运行输出（实测）：1 2 3 4 5

while 的套路是**三件套**：

1. **初始化**：进循环前先把计数器设好（int n = 1;）；
2. **条件**：while（条件）——为真继续，为假退出；
3. **更新**：循环体里改计数器（n++）——**忘了它 = 死循环**。

> **新手坑 1：死循环。**条件永远为真就无限循环（程序"卡住"）。急救：按 Ctrl+C 中断。写 while 前先问自己：这个条件**有办法变假**吗？

6.2 for 循环：三件套写在一行

for 把「初始化、条件、更新」三个部件放进一对括号，中间用分号隔开，是**计数循环**最常用的写法：

```
/* ch6_02_for.c — for 求和 1+2+...+10 */
#include <stdio.h>

int main(void)
```

```
{
    int sum = 0;
    for (int i = 1; i <= 10; i++) {   /* 初始化; 条件; 更新 */
        sum += i;
    }
    printf("1+2+...+10 = %d\n", sum);
    return 0;
}
```

运行输出（实测）：`1+2+...+10 = 55`

部件	说明	示例
初始化	进循环前执行一次，通常声明计数器	`int i = 1`
条件	每次进入前检查，为真继续	`i <= 10`
更新	每轮循环体结束后执行	`i++`

> **口诀：**「从几开始，到几为止，每次怎么变」——`for（开始；条件；变化）`。C99 起可以在 `for` 里直接声明 `int i`，循环结束 `i` 就"消失"。

6.3 do-while：至少做一次

有时候你想「先干一次，再问要不要继续」——用 `do-while`。它和 `while` 唯一的区别：**条件检查在循环体后面**，所以循环体**至少执行一次**。

```
/* ch6_03_dowhile.c — do-while 至少执行一次 */
#include <stdio.h>

int main(void)
{
    int n = 10;
    do {
        printf("执行了一次，n = %d\n", n);
        n++;
    } while (n <= 3);       /* 条件为假，但循环体已执行过 */
    return 0;
}
```

运行输出（实测）：`执行了一次，n = 10`

6.4 break 和 continue：循环里的"刹车"和"跳过"

- `break`：立即**跳出整个循环**（不再管条件）；

- `continue`：跳过**本次**循环体剩余部分，直接进下一轮。

```
/* ch6_04_breakcont.c — 找第一个能被 7 整除的数（break）/ 只打印偶数（continue）*/
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 100; i++) {
        if (i % 7 == 0) { printf("第一个 7 的倍数：%d\n", i); break; }
    }
    for (int i = 1; i <= 6; i++) {
        if (i % 2 != 0) continue;   /* 奇数跳过，不打印 */
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

运行输出（实测）：第一个 7 的倍数：7 2 4 6

6.5 嵌套循环：循环套循环（九九乘法表）

循环体里还可以是循环——叫**嵌套循环**。外层走一步，内层走一整圈。经典例子是打印九九乘法表：

```
/* ch6_05_table.c — 九九乘法表（上半部分）*/
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 3; i++) {          /* 外层：行 */
        for (int j = 1; j <= i; j++) {      /* 内层：列，只到 i */
            printf("%d*%d=%d  ", j, i, j * i);
        }
        printf("\n");
    }
    return 0;
}
```

运行输出（实测）：

```
1*1=1
1*2=2  2*2=4
1*3=3  2*3=6  3*3=9
```

> **读法：**i=1 时内层跑 1 次；i=2 时内层跑 2 次；i=3 时跑 3 次——"外层控制行，内层控制列"。

6.6 新手坑集中营（本章最容易错的 4 件事）

1. **死循环**：忘更新计数器或条件恒真。`while (1)` 是故意的死循环（嵌入式里常用），别的都是 bug。
2. **`while` 后面多加分号**：`while (n <= 5); { ... }`——分号让循环体变空语句，条件一直不变 → 死循环（或一次都不执行）。
3. **条件写成赋值**：`while (n = 5)` 是把 5 赋给 n（条件恒真）；判断相等要用 `==`。
4. **for 的三段用逗号**：`for (int i=1, i<=5, i++)` 是语法错误——三段之间是**分号**，不是逗号。

6.7 一分钟自测（答案见练习册）

> **说出结果：**
> ① `for (int i=0; i<3; i++) printf("%d", i);` 打印 ____
> ② `int n=5; while (n<5) n++; printf("%d", n);` 打印 ____
> ③ 下面这段会打印几次 "hi"？`for (int i=0; i<5; i++) { if (i==2) continue; printf("hi "); }`

6.8 练习指引

做完本章内容后，去练习册完成 Ch6 配套练习；同时打开《C Primer Plus（第6版）》第 6 章，完成**复习题 2、3、6**（题号对应主教材）。改错题是最有价值的——先自己找，再看答案。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译，每句「运行输出（实测）」均已实跑核对。原书题目请以正版教材为准。

C 语言伴读讲义 · 第 3 批

Ch7 分支和跳转

对应《C Primer Plus（第6版）》第 7 章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：goto、条件运算符嵌套

7.1 if / else：让程序"做选择"

循环让程序会**重复**，分支让程序会**判断**。最基础的是 `if`：条件为真做一件事，否则做另一件事。

```
/* ch7_01_if.c — if / else 判断成绩是否及格 */
#include <stdio.h>

int main(void)
{
    int score = 76;
    if (score >= 60) {
        printf("及格，继续加油！\n");
    } else {
        printf("不及格，需要补考\n");
    }
    return 0;
}
```

运行输出（实测）：及格，继续加油！

> **新手坑 1：== 和 = 写反。**判断相等要用 `==`（两个等号）。写 `if (score = 60)` 是把 60 赋给 score——结果恒为"真"，编译器通常只给 warning，程序逻辑完全错。这是 C 初学者最经典的一坑。

7.2 else if：多分支"排队判断"

判断不止两种情况时，用 `else if` 一路排下去，从上到下第一个为真的分支执行，其余跳过。

```
/* ch7_02_elseif.c — 成绩等级 */
#include <stdio.h>

int main(void)
{
    int score = 85;
    if (score >= 90)      printf("A\n");
    else if (score >= 80) printf("B\n");
    else if (score >= 70) printf("C\n");
```

```
    else if (score >= 60) printf("D\n");
    else                  printf("F\n");
    return 0;
}
```

运行输出（实测）：B

> **读法：**从上往下问"够不够 90？够不够 80？"——第一个"够"就停。条件有顺序：先写大的再写小的（如果先写 `>=60` 再写 `>=90`，90 分会被误判成 D）。

7.3 switch-case：按值"一键分路"

当判断的是"一个变量的值等于几"（而不是大小比较），`switch` 比一串 `else if` 更清晰。注意：**每个 case 末尾都要** `break`，否则会"掉下去"执行下一个 case。

```
/* ch7_03_switch.c — 菜单选择 */
#include <stdio.h>

int main(void)
{
    int choice = 2;
    switch (choice) {
        case 1: printf("打开文件\n"); break;
        case 2: printf("保存文件\n"); break;
        case 3: printf("退出程序\n"); break;
        default: printf("无效选择\n");
    }
    return 0;
}
```

运行输出（实测）：保存文件

> **新手坑 2：case 里忘写 break。**写 `case 1: printf("打开");` 不加 `break`，选 1 会把"保存文件"也打出来（fall-through 掉穿）。多数情况下这是 bug。

7.4 逻辑运算符：&& || !

把多个条件组合起来：

运算符	含义	示例
`&&`	并且（两边都为真才真）	`score>=60 && score<100`
`||`	或者（一边为真就真）	`ch=='y' || ch=='Y'`

!	取反（真变假，假变真）	!done

```c
/* ch7_04_logic.c — 判断闰年（能被4整除且不被100整除，或能被400整除）*/
#include <stdio.h>

int main(void)
{
    int year = 2024;
    if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
        printf("%d 是闰年\n", year);
    else
        printf("%d 不是闰年\n", year);
    return 0;
}
```

运行输出（实测）：2024 是闰年

7.5 字符输入：getchar 与"字符判断"

程序还能读单个字符并判断。常用的 getchar() 读一个字符，返回它的 ASCII 值。字符在 C 里就是数字，可以直接比较。

```c
/* ch7_05_getchar.c — 读字符判断是数字还是字母 */
#include <stdio.h>

int main(void)
{
    printf("输入一个字符：");
    char ch = getchar();
    if (ch >= '0' && ch <= '9')
        printf("是数字\n");
    else if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z'))
        printf("是字母\n");
    else
        printf("是其他字符\n");
    return 0;
}
```

运行输出（实测，输入 5）：是数字

> **原理：**字符 '0'~'9' 的 ASCII 是 48~57，'a'~'z' 是 97~122，'A'~'Z' 是 65~90（Ch3 讲过）。ch >= '0' 就是拿数字比较，无需背 ASCII 码表。

7.6 新手坑集中营（本章最容易错的 4 件事）

1. == **写成** =（见 7.1）——判断相等用双等号。
2. **switch 忘 break**（见 7.3）——case 会掉穿。
3. **悬空 else**：`else` 跟**最近的**那个 `if` 配对，不是跟缩进对齐的那个。拿不准就加大括号。
4. **else if 顺序颠倒**（见 7.2）——先写宽条件会把后面的窄条件吃掉。

7.7 一分钟自测（答案见练习册）

说出结果：

① `int n=5; if (n=3) printf("A"); else printf("B");` 打印 _____

② `int x=85; if (x>=60) printf("P"); else if (x>=80) printf("G");` 打印 _____

③ `char c='b'; if (c=='a' || c=='b') printf("yes");` 打印 _____

7.8 练习指引

做完本章内容后，去练习册完成 Ch7 配套练习；同时打开《C Primer Plus（第6版）》第 7 章，完成**复习题 2、4、6**（题号对应主教材）。改错题是最有价值的——先自己找，再看答案。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译，每句「运行输出（实测）」均已实跑核对。原书题目请以正版教材为准。

C 语言伴读讲义 · 补全章

Ch9 函数

对应《C Primer Plus（第6版）》第 9 章 · 本章目标：40 分钟读懂 + 60 分钟练完 · 前置：Ch1–Ch7 · 学完本章，Ch10 的数组与指针才能继续

9.1 先看一个真实场景：为什么需要函数

你写了一个「把 10 个 LED 依次点亮」的程序，现在要再点亮 20 个、30 个——把点亮逻辑复制三遍？代码会越堆越长，改一个细节要改三处。函数就是**给一段代码起名字、随叫随到**的工具：

- **复用**：同一段逻辑写一次，需要时调用多次；
- **分工**：大程序拆成小函数，各管一段，出错好定位；
- **可读**：main 里全是「做什么」（turn_on_led(3)），而不是「怎么做」。

电子信息专业的视角：驱动库、算法库（FFT、滤波）全部是「函数集合」。你之后读学长代码、用厂商 SDK，第一步就是找它的函数原型。

9.2 函数的定义：返回值类型 + 名字 + 参数 + 函数体

函数定义长这样（和 main 同级的兄弟函数）：

```
/* ch9_01_add.c — 最简单的函数：两数相加 */
#include <stdio.h>

int add(int a, int b)       /* 返回类型 int；参数 a、b 都是 int */
{
    return a + b;           /* return 把结果交回给调用者 */
}

int main(void)
{
    int r = add(3, 5);      /* 调用：实参 3 和 5 传给形参 a 和 b */
    printf("%d\n", r);      /* 输出 8 */
    return 0;
}
```

编译：`gcc -Wall -Wextra -o add.exe ch9_01_add.c` 运行：`.\add.exe` → 8

部件	说明	示例

返回类型	函数交回的值是什么类型；不需要交回就用 `void`	`int` / `void`
函数名	动词化命名，让别人一看就知道干什么	`add` / `turn_on_led`
参数列表	函数需要的「输入」；没有就用 `void`	`(int a, int b)` / `(void)`
函数体	{} 里的语句，最后通常 return	`{ return a + b; }`

> **新手坑 1：形参和实参是两个东西。**定义里写的是**形参**（a、b 是占位符），调用时传的是**实参**（3、5 是真值）。调用那一刻，实参的值被**复制**给形参——函数内部改形参，外面的实参不受影响（9.4 细讲）。

9.3 函数原型：先声明，后使用

编译器读代码是**从上到下**的。如果 main 在 add 前面，编译器看到 `add(3, 5)` 时还不知道 add 是什么 → 报「隐式声明」错误（C99 之后直接拒绝编译）。解决办法：在调用之前放一行**函数原型**：

```
/* ch9_02_proto.c — 函数原型先行 */
#include <stdio.h>

int add(int a, int b);      /* 函数原型：只声明签名，不写函数体 */

int main(void)
{
    printf("%d\n", add(3, 5));
    return 0;
}

int add(int a, int b)       /* 函数定义可以放后面 */
{
    return a + b;
}
```

> **习惯**：所有函数原型集中在 main 前面（或放头文件），函数定义随意排。这也是你读别人代码时的第一站。

9.4 参数传递：值传递（swap 为什么不行）

参数是**值传递**——传进去的是副本。经典的「交换两个数」陷阱：

```c
/* ch9_03_swap.c — 为什么 swap 没生效？ */
#include <stdio.h>

void swap(int a, int b)    /* 形参 a、b 是实参的副本 */
{
    int t = a;
    a = b;
    b = t;                 /* 只交换了副本 */
}

int main(void)
{
    int x = 1, y = 2;
    swap(x, y);
    printf("x=%d y=%d\n", x, y);   /* 输出 x=1 y=2，没换！ */
    return 0;
}
```

实测输出：x=1 y=2——swap 里的 a、b 是 x、y 的复制品，交换副本不影响原件。

> **新手坑 2：想改外面的变量，要么传地址（指针，Ch10 学），要么让函数 return 新值再赋值。**目前阶段：需要「带出结果」就 return，如 x = add(x, 5);。

9.5 返回值与 void：函数能交回什么

return 表达式; 把结果交回调用处；void 函数不交回值（但可以 return; 提前结束）：

```c
/* ch9_04_return.c — 返回值与提前返回 */
#include <stdio.h>

int abs_val(int n)         /* 绝对值 */
{
    if (n < 0) return -n;  /* 提前返回 */
    return n;
}

void report(int n)         /* void：只做事，不交回 */
{
    printf("abs(%d) = %d\n", n, abs_val(n));
}

int main(void)
{
    report(-7);            /* 输出 abs(-7) = 7 */
    report(4);             /* 输出 abs(4) = 4 */
```

```
    return 0;
}
```

> **返回类型要对上**：return 的类型和函数声明的返回类型不一致，编译器会警告（-Wall 会提醒）。double 函数里 return int 值 → 自动转 double，但反过来 int 函数里 return double 会**截断**（丢小数）。

9.6 函数与数组：传数组 = 传首地址（const 保护）

把整个数组复制给函数太浪费，C 的做法是**只传数组首地址 + 长度**（指针细节 Ch10 展开，这里先会用）：

```
/* ch9_05_array.c — 数组求和函数（Ch10 会用到同款写法）*/
#include <stdio.h>

int sum_array(const int arr[], int n)   /* const：承诺不修改数组内容 */
{
    int s = 0;
    for (int i = 0; i < n; i++)
        s += arr[i];
    return s;
}

int main(void)
{
    int a[] = {2, 4, 6, 8};
    printf("%d\n", sum_array(a, 4));    /* 输出 20 */
    return 0;
}
```

实测输出：20。`arr` 形参里写 `const int arr[]` 是告诉编译器「我只读不改」——误改 `arr[i]` 会直接报错，防止手滑。

> **新手坑 3：函数里 sizeof(arr) 不是数组大小！**形参 `int arr[]` 本质是指针，`sizeof(arr)` 是指针大小（8 字节，64 位机），不是数组字节数。所以必须**同时传长度 n**。这是 C 语言的经典陷阱，面试也爱考。

9.7 递归：函数调用自己（先看懂再写）

递归 = 函数体里调用它自己，配合「终止条件」结束。经典阶乘：

```
/* ch9_06_fact.c — 递归求阶乘 */
#include <stdio.h>

long fact(int n)
{
    if (n <= 1) return 1;   /* 终止条件：不再往下调 */
    return n * fact(n - 1); /* 自己调用自己，规模缩小 */
```

```
}

int main(void)
{
    printf("%ld\n", fact(5));   /* 输出 120 */
    return 0;
}
```

实测输出：120（5×4×3×2×1）。递归执行路径：fact(5) → 5×fact(4) → 5×4×fact(3) → … → 5×4×3×2×1。

> **递归三要素**：①终止条件（不满足就无限调 → 栈溢出崩溃）；②每次调用规模变小；③能把「缩小后的结果」拼回大结果。写不出递归就用循环，二者等价，别硬撑。

9.8 新手坑集中营（本章最容易错的 4 件事）

- **1. 漏函数原型**：main 在前、函数定义在后，中间没有原型 → 编译报「隐式声明」错误。报错先看是不是函数顺序问题。
- **2. 函数里 sizeof(数组形参)**：拿到的是指针大小不是数组大小——数组参数必须另传长度。
- **3. 忘记 return / 返回类型不匹配**：int 函数漏 return 是**未定义行为**（可能返回垃圾值）；-Wall 会警告「control reaches end of non-void function」——看到立刻补。
- **4. 想用函数改外部变量却用值传递**：swap(x,y) 换不了——要么 return 新值，要么等 Ch10 学指针再传地址。

9.9 一分钟自测（先答后揭，答案见练习册）

Q1：函数原型和函数定义的区别是什么？
Q2：void 函数可以写 return; 吗？
Q3：int sum(const int a[], int n) 里的 const 有什么用？
Q4：为什么 swap(int a, int b) 换不了外面的两个变量？
Q5：递归必须有的两个条件是什么？

9.10 练习指引

打开「练习册 · Ch9」：先做 **9-1 填空**（热手）→ **9-2 改错**（先自己找错再对答案，最有价值）→ **9-3/9-4 编程**（自己写，参考代码在参考答案）→ **9-5 专业场景**（写一个 LED 灯效函数，电子信息味）。写完对照参考答案，重点看「常见错法」。

C 语言伴读讲义 · 教案 v2 样板

Ch10 数组和指针

对应《C Primer Plus（第6版）》第 10 章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：多维数组深度用法、`int (*p)[5]` 这类复杂声明

10.0 本章定位与取舍

学完这一章你会写什么：用数组存一组数据、用指针遍历它、写一个能改到"外面数组"的函数。

必看：10.2 数组名不是变量 · 10.5 为什么 `a[i]` 等于 `*(a+i)` · 10.7 函数传数组

可跳过：10.6 的 `*p++`（第一次读知道有这么回事就行，写代码时回头再看）

以后再看：多维数组、指向数组的指针、动态内存（Ch12 会讲 `malloc`）

本章在课程里的位置：Ch4 埋过一句"数组名是地址"的伏笔，这一章兑现它；Ch9 函数与本章合用，才能写出"能改到外面"的函数。

10.1 先看一个真实场景：ADC 采样

单片机每隔一段时间读一次 ADC（模数转换），把 12 位的结果存起来：

```
/* 想象中的数据长这样，不是要你现在写 */
第 1 次采样：2048
第 2 次采样：2011
第 3 次采样：2056
…一共采 64 次
```

如果只用单个变量，你得声明 `adc0` 到 `adc63` 六十四个名字——这显然不行。

你需要的是「**一排连续的、同类型的格子**」，能用一个名字加编号访问。这就是**数组**。

10.2 数组：一排连续的格子

`int a[5];` 一次申请 5 个 `int` 的格子，用 `a[0]` 到 `a[4]` 访问。**下标从 0 开始，不是 1**——第一个格子叫 `a[0]`。

```
/* ch10_01_array.c */
#include <stdio.h>

int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    int i;
```

```
    for (i = 0; i < 5; i++)
        printf("a[%d] = %d\n", i, a[i]);
    return 0;
}
```

编译：gcc -std=c11 -Wall -Wextra -o a.exe ch10_01_array.c ｜ 运行输出（实测）：

```
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
a[4] = 50
```

> **新手坑 · 基础：越界不报错。**a[5] 编译器**一句话都不说**。C 不检查下标，它只是"悄悄读/写了数组后面那块内存"。轻则读到垃圾值，重则改坏别的变量。写循环边界时多看一眼：i < 5 不是 i <= 5。

10.3 数组名是地址，但不是变量

这一节是本章**最容易学错**的地方。先记结论，再看三组实验：

> **一句话模型：数组名是地址，但它不是变量——不能给它赋值，也不能自增。**

实验一：数组名就是首格地址

```
/* ch10_02_addr.c */
#include <stdio.h>

int main(void)
{
    int a[3] = {7, 8, 9};

    printf("a     = %p\n", (void*)a);
    printf("&a[0] = %p\n", (void*)&a[0]);
    printf("&a[1] = %p\n", (void*)&a[1]);
    return 0;
}
```

运行输出（实测，地址每次运行会变，看差值）：

```
a     = 0000005445BFF964
&a[0] = 0000005445BFF964   ← 和 a 完全相同
&a[1] = 0000005445BFF968   ← 比 a[0] 大 4 = sizeof(int)
```

实验二：sizeof 在 main 里和函数里不一样

这是"数组名不是指针"最直接的证据。同一个 a，出了 main 就"变小"了：

```
/* ch10_02_sizeof.c */
#include <stdio.h>

void show(int arr[])
{
    printf("函数里 sizeof(arr)  = %zu\n", sizeof(arr));
    printf("函数里 sizeof(*arr) = %zu\n", sizeof(*arr));
}

int main(void)
{
    int a[5] = {1, 2, 3, 4, 5};

    printf("main 里 sizeof(a)   = %zu\n", sizeof(a));
    printf("main 里元素个数     = %zu\n", sizeof(a) / sizeof(a[0]));

    show(a);
    return 0;
}
```

运行输出（实测）：

```
main 里 sizeof(a)   = 20   ← 5 个 int，整个数组
main 里元素个数     = 5
函数里 sizeof(arr)  = 8    ← 只是指针的大小（64 位）
函数里 sizeof(*arr) = 4    ← 一个 int
```

> **编译器会主动提醒你这件事。**上面这段代码用 -Wall -Wextra 编译时，gcc 会报一条警告（实测原文）：
> warning: 'sizeof' on array function parameter 'arr' will return size of 'int *' [-Wsizeof-array-argument]
> **这条警告就是编译器在替你回答"参数到底是不是数组"。**所以：**函数里永远不要用 sizeof(arr) 求数组长度**，要把长度当参数传进去（见 10.7）。

实验三：给数组名自增，编译器直接拦下

❌ **错例（不要照抄）**

```
/* debug_ch10_arrayname.c */
int a[3] = {1, 2, 3};
a++;              /* 想让 a 往后走一格 */
```

编译报错（实测原文）：

✅ **对例：让指针走**

```
/* 想移动就用指针 */
int a[3] = {1, 2, 3};
int *p = a;
p++;              /* p 现在指向 a[1] */
```

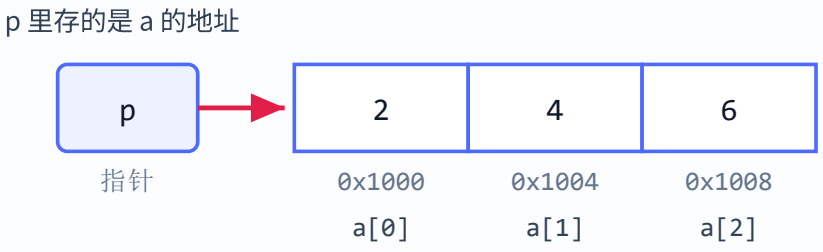
```
error: lvalue required as
increment operand
```

人话翻译：++ **要求左边是"能被赋值的变量"，而数组名不是。**

指针 p 是变量，可以自增；数组名 a 不是。

新手坑 · 基础：别把"数组名"当成"指针"。它们的**值**常常相同（都指向首格），但**性质**不同： 数组名不能被赋值、不能自增、sizeof 是整个数组的大小；指针这三件事都能做。

10.4 指针：专门存地址的变量



内存模型：**数组 a 是一排连续的格子**，地址 0x1000 / 0x1004 / 0x1008（每个 int 占 4 字节）；**指针 p 自己也是一块内存**，里面存着 0x1000 这个**地址值**。（地址为示意，实际每次运行不同）

两个配套运算符，C 里最基础也最容易搞反的一对：

- & **（取地址）**：p = &n; 把 n 的地址存进 p。
- * **（解引用）**：*p 表示"p 指向的那个变量"。*p = 5; 等价于给 n 赋值 5。

```c
/* ch10_03_pointer.c */
#include <stdio.h>

int main(void)
{
    int n = 42;
    int *p = &n;      /* p 存 n 的地址 */

    printf("n = %d\n", n);
    printf("*p = %d\n", *p);   /* 通过指针读 n */
    *p = 100;                  /* 通过指针改 n */
    printf("改后 n = %d\n", n);
    return 0;
}
```

运行输出（实测）：

```
n = 42
*p = 42
改后 n = 100
```

> **新手坑 · 基础：野指针。**`int *p; *p = 5;` —— `p` 没初始化，指向哪里未知，往里写就是往一块不知道是谁的内存里写。
> 实测（gcc 16.1.0）：单独编译这一行会得到 `warning: 'p' is used uninitialized [-Wuninitialized]`（本机开不开 `-O2` 都会报）。
> **但不要用"有没有警告"来判断野指针**——如果同一段代码里还有别的硬错误，编译器可能根本走不到这条分析。

10.5 a[i] 就是 *(a+i)：为什么

本章最核心的一个等式：

```
a[i]  ==  *(a + i)
```

它**不是约定，是定义**。理解它需要先回答一个问题：`a + 1` **到底加了多少？**

先看实验：同一个 +1，跳的距离不一样

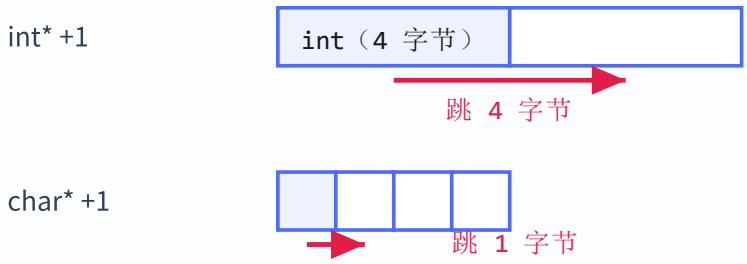
```
/* ch10_06_ptrstep.c */
#include <stdio.h>

int main(void)
{
    int  a[3] = {0x11111111, 0x22222222, 0x33333333};
    int  *pi = a;
    char *pc = (char *)a;

    printf("int*  pi + 1   = %p   跳 %ld 字节\n",
           (void *)(pi + 1), (long)((char *)(pi + 1) - (char *)pi));
    printf("char* pc + 1   = %p   跳 %ld 字节\n",
           (void *)(pc + 1), (long)((pc + 1) - pc));
    return 0;
}
```

运行输出（实测，地址每次运行会变，看"跳几字节"）：

```
int*  pi + 1   = 000000299C3FFE08   跳 4 字节
char* pc + 1   = 000000299C3FFE05   跳 1 字节
```



指针加法按**它指向的类型**缩放：`int*` 跳 `sizeof(int)`=4 字节，`char*` 跳 1 字节。所以 `a + i` 是"从数组开头往后数 **i 个元素**"的地址，不是"i 个字节"。

于是等式就成立了

1. `a` 是首格地址。
2. `a + i` 是"往后数 i 个元素"的地址——**因为是按元素缩放，所以正好落在 `a[i]` 上**。
3. `*(a + i)` 就是"取那个地址上的值"＝`a[i]`。

所以**下标只是指针运算的语法糖**：`a[i]` 是写给人类看的，编译器看到的就是 `*(a+i)`。理解这一点，你就同时理解了：为什么数组名能当指针用、为什么指针也能用 `p[i]` 下标。

```
/* ch10_04_equiv.c */
#include <stdio.h>

int main(void)
{
    int a[4] = {2, 4, 6, 8};
    int *p = a;

    printf("a[2] = %d, *(a+2) = %d\n", a[2], *(a + 2));
    printf("p[2] = %d, *(p+2) = %d\n", p[2], *(p + 2));
    return 0;
}
```

运行输出（实测）：

```
a[2] = 6, *(a+2) = 6
p[2] = 6, *(p+2) = 6
```

10.6 *p++ 还是 (*p)++

这两个写法只差一对括号，但做的事**完全不同**。先看实测，再看记忆法：

```
/* ch10_07_ptrincr.c */
#include <stdio.h>

int main(void)
{
    int a[3] = {10, 20, 30};
    int *p = a;
    int x, y;

    x = *p++;   /* 先取出 *p，再让 p 往后走一格 */
    printf("x = %d, *p = %d, p 指向 a[%ld]\n", x, *p, (long)(p - a));

    p = a;      /* 复位 */
    y = (*p)++; /* 先取出 *p，再把那一格内容加 1；p 不动 */
    printf("y = %d, a[0] = %d, p 仍指向 a[%ld]\n", y, a[0], (long)(p - a));
    return 0;
}
```

运行输出（实测）：

```
x = 10, *p = 20, p 指向 a[1]
y = 10, a[0] = 11, p 仍指向 a[0]
```

写法	谁变了	变多少	结果
*p++	**指针 p**	往后走 1 个元素	遍历数组用
(*p)++	**p 指向的内容**	内容 +1	给数组元素计数累加用

> **记忆法：看 ++ 贴着谁。**++ 紧贴 p（*p++ 里的 p++）就动指针；++ 紧贴括号（(*p)++）就动内容。

状态追踪：用 *p++ 遍历数组

把每一步的变量值摊开来看，比读文字清楚得多：

```
/* ch10_08_trace.c 的循环体 */
for (i = 0; i < 4; i++)
    sum += *p++;   /* 取值 + 指针前进，一步完成 */
```

轮次 i	执行前 *p	执行前 p 指向	sum 累加后	执行后 p 指向
0	2	a[0]	2	a[1]
1	4	a[1]	6	a[2]

2	6	a[2]	12	a[3]
3	8	a[3]	20	a[4]（末尾之后）

运行输出（实测）：

```
sum = 20
循环结束后 p - a = 4（p 已走到末尾之后再一格）
```

> **新手坑 · 基础：循环结束后 p 已经不在数组里了。** `p - a = 4`，而数组只有 `a[0]`–`a[3]`。此时**不要解引用** `*p`。规范的写法是在循环里用 `a[i]`，或者循环后重新 `p = a;`。

10.7 函数传数组：传的是地址

函数形参写成 `int arr[]`，实际拿到的是**指向数组开头的指针**——数组不会被复制。所以函数里改数组，外面的数组会跟着变。

```c
/* ch10_05_func.c */
#include <stdio.h>

int sum(const int arr[], int n)   /* const 承诺不修改数组内容 */
{
    int i, total = 0;
    for (i = 0; i < n; i++)
        total += arr[i];
    return total;
}

int main(void)
{
    int a[5] = {1, 2, 3, 4, 5};
    printf("总和 = %d\n", sum(a, 5));
    return 0;
}
```

运行输出（实测）：

```
总和 = 15
```

> **为什么 n 必须单独传？** 回到 10.3 的实验二：函数里 `sizeof(arr)` 只有 8，量不出数组有多长。所以「数组 + 长度」要成对传，这是 C 的固定套路。
> **为什么写 `const`？** 函数不打算改数组时就写 `const int arr[]`，编译器会拦住你误改——这是给自己设的护栏。

10.8 新手坑：基础与扩展

级别	坑	怎么躲
基础	下标越界（a[5] 合法访问 5 个元素的数组）	循环写 i < n；C 不检查越界
基础	函数里 sizeof(arr) 当数组长度	长度当参数传；看 -Wsizeof-array-argument 警告
基础	给数组名自增 / 赋值	要移动就用指针；数组名不是变量
基础	& 与 * 搞反	p = &n 存地址；*p = 5 改内容
基础	以为传数组是复制	函数改形参数组＝改外面；要副本得自己拷
扩展	循环结束后解引用 *p	p 已到末尾之后；循环后重新 p = a
扩展	把 a 和 &a 当成一回事	值相同但类型不同（&a+1 跳整个数组）

10.9 一分钟自测（先答后揭）

先在纸上写下你的答案，再点开对照——**直接看答案等于没做**。

▾ ① `int a[5] = {1,2,3,4,5}; printf("%d", a[5]);` **会输出 5 吗？**

正解：不会。a[5] 越界了——合法下标只有 0–4。它读到的是数组后面那块内存，值是**不确定的**（可能正好是别的变量，也可能是垃圾值）。

常见错解："会，因为 a[5] 是第 5 个元素。" ← 错在把"第 5 个"当成 a[5]；第一个元素是 a[0]，第 5 个是 a[4]。

为什么这条最容易错：日常语言从 1 数起，C 从 0 数起；而且越界**不报错**，写错的人常常"看起来能跑"。

▾ ② `int *p; *p = 10;` **是合法写法吗？**

正解：语法合法，但行为未定义——不能这么写。p 没初始化，里面是残留值，*p = 10 等于往一块不知道属于谁的内存里写。

常见错解："编译能过就是合法。" ← 错在混淆"编译通过"与"行为正确"。C 里这两件事经常不一致，这也是本章反复强调"看编译器警告"的原因。

正确做法：先让指针有明确指向（int n; int *p = &n;），或指向动态内存（Ch12）。

▾ ③ **函数形参写** `int arr[]`**，它实际是什么类型？**sizeof(arr) **是多少？**

正解：实际是 int *arr；sizeof(arr) **是指针大小（本机 64 位 = 8），不是整个数组。**

常见错解："是数组，sizeof 应该等于元素个数 × 4。" ← 错在把形参当成了数组本体。数组名作实参时

> 会"退化"成指向首元素的指针，函数拿到的是地址，不是副本。
>
> **交叉验证：**本机实测 `main` 里 `sizeof(a)=20`、函数里 `sizeof(arr)=8`（见 10.3 实验二）。

10.10 练习指引

四级阶梯，按顺序做（做完再去练习册对答案）：

1. **照抄改数**：把 `ch10_01_array.c` 的初值改成你自己的 5 次 ADC 采样值，打印出来。
2. **自己写逻辑**：写一个函数 `int max_of(const int arr[], int n)` 返回最大值，并在 `main` 里调用验证。
3. **组合两个知识点**：用 10.6 的 `*p++` 写法重写上面的遍历，并对照 `p - a` 观察指针最终停在哪。
4. **专业场景**：用数组存 64 次 12 位 ADC 采样，算出平均值与峰峰值（最大值 − 最小值）——先想清楚"为什么函数里量不出数组长度"。

本章配套练习见《练习册 · Ch10》。主教材复习题建议完成 **2、4、6** 三题。

下一章预告：Ch12 会讲动态内存 `malloc/free`——那是"数组长度在运行时才确定"时的办法。本章的 `a[i] == *(a+i)` 与 `p - a` 在那里会再用到。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均用 `gcc -std=c11 -Wall -Wextra` 编译并实跑，输出与文中标注一致（唯一例外见 10.3 实验二，那条警告是本节的教学内容本身）。原书题目请以正版教材为准。

C 语言伴读讲义 · 补全章

Ch11 字符串

对应《C Primer Plus（第6版）》第 11 章 · 本章目标：40 分钟读懂 + 60 分钟练完 · 前置：Ch4（printf/scanf）+ Ch9（函数）· 学完本章，fgets/strlen/strcpy/strcmp 全部落地

11.1 先看一个真实场景：程序要处理「名字」

你写一个考勤程序，要读入同学名字、统计长度、比较两个名字是否相同。C 语言里「名字」不是一种内置类型，而是**字符数组 + 结尾的 \0**。这一章就是把这套东西讲透——之后读传感器字符串指令（串口 AT 指令）、解析日志行，全靠它。

11.2 字符串是什么：char 数组 + '\0'

字符串字面量 "hi" 实际是 3 个字节：'h' 'i' '\0' ——末尾那个 \0（值为 0 的字符）叫**空字符**，是字符串的「句号」，函数靠它知道到哪结束：

```
/* ch11_01_str.c — 字符串就是 char 数组 */
#include <stdio.h>

int main(void)
{
    char name[] = "hi";          /* 自动变成 {'h','i','\0'}，3 个元素 */
    char word[8] = {'h', 'e', 'y', '\0'};   /* 手动：必须有 \0 结尾 */

    printf("%s | %s\n", name, word);
    printf("name 占 %zu 字节\n", sizeof(name));   /* 输出 3（含 \0）*/
    return 0;
}
```

实测输出：hi | hey name 占 3 字节。注意 sizeof(name) 是 3（含 \0），而 strlen(name) 是 2（不含）。

> **数组长度 ≥ 内容长度 + 1**：留一位给 \0。char s[5] 最多存 4 个字符 + \0。

11.3 字符串的读写：printf / scanf / fgets

输出用 %s；输入有两个选择，**新手必用 fgets**：

```
/* ch11_02_io.c — scanf 遇空格停 vs fgets 读整行 */
#include <stdio.h>
```

```c
int main(void)
{
    char a[32], b[32];
    int c;

    printf("输入一行（含空格）：");
    scanf("%s", a);                /* 危险：读到空格就停，且不检查长度 */
    while ((c = getchar()) != '\n' && c != EOF) {}  /* 清掉 scanf 留在缓冲区的回车；EOF 时退出，防重定向死循环 */

    printf("再输入一行：");
    fgets(b, sizeof(b), stdin);   /* 安全：读一整行，最多 31 字符+ \0 */

    printf("scanf 读到：%s\n", a);
    printf("fgets 读到：%s", b);  /* fgets 会保留结尾换行 */
    return 0;
}
```

新手坑 1：scanf("%s") 两个雷：①遇到空格就停（"Li Ming" 只读到 "Li"）；②不限制长度，输入超长会**越界写坏内存**（缓冲区溢出）。所以读一行用 `fgets`，别用 scanf %s。

新手坑 2：fgets 会把结尾 \n 也存进来。想去掉它：`size_t len = strlen(b); if (len > 0 && b[len-1] == '\n') b[len-1] = '\0';`——这是处理 fgets 的标配三行。

新手坑 3：清缓冲区必须防 EOF。上面 `while ((c = getchar()) != '\n' && c != EOF)` 里的 `c != EOF` 不能省——若输入被重定向或结束（如 `./a.exe < data.txt`、判题系统喂数据），`getchar()` 会一直返回 EOF，不带 EOF 判断就会**死循环**。用 `int c` 接收（不能是 `char`，EOF 是负值）。

11.4 字符串函数：strlen / strcpy / strcmp / strcat

函数	作用	示例
`strlen(s)`	长度（不含 \0）	strlen("hi") → 2
`strcpy(dst, src)`	把 src 复制到 dst	strcpy(b, a);
`strcmp(a, b)`	比较内容：0 相等 / <0 a 小 / >0 a 大	if (strcmp(a, b) == 0)
`strcat(dst, src)`	把 src 接到 dst 末尾	strcat(b, "!");

这些函数在 `#include <string.h>` 里：

```c
/* ch11_03_funcs.c — 字符串函数全家桶 */
#include <stdio.h>
#include <string.h>

int main(void)
{
    char a[] = "Li", b[32] = "";

    strcpy(b, a);                       /* 复制：b = "Li" */
    strcat(b, " Ming");                 /* 拼接：b = "Li Ming" */

    printf("%s（长度 %zu）\n", b, strlen(b));
    printf("比较 a 和 \"Li\"：%d\n", strcmp(a, "Li"));   /* 0=相等 */
    return 0;
}
```

实测输出：Li Ming（长度 7） 比较 a 和 "Li"：0。

> **新手坑 3：strcmp 比的是内容，不是地址**。`if (a == "Li")` 比较的是两个地址（永远不相等），必须 `strcmp(a, "Li") == 0`。同理 `==` 不能用来比字符串。

> **新手坑 4：strcpy/strcat 不检查空间**。目标数组太短会越界写坏内存。稳妥做法：目标数组开大一点 + 用 `strncpy/strncat`（带长度版本）。

11.5 遍历字符串：逐个处理字符

配合循环可以数单词、找字符、统计字母：

```c
/* ch11_04_count.c — 统计字母个数（\0 是循环的终点）*/
#include <stdio.h>
#include <string.h>

int main(void)
{
    char line[] = "Hello, LZU 2026!";
    int letters = 0;

    for (int i = 0; line[i] != '\0'; i++)
        if ((line[i] >= 'a' && line[i] <= 'z') || (line[i] >= 'A' && line[i] <= 'Z'))
            letters++;

    printf("字母数：%d\n", letters);   /* H e l l o L Z U = 8 */
    return 0;
}
```

实测输出：字母数：8。循环条件 line[i] != '\0' 是遍历字符串的标准写法；也可以用 i < strlen(line)（每次重算长度，稍慢）。

11.6 新手坑集中营（本章最容易错的 4 件事）

- **1. scanf("%s") 遇空格停 + 不查长度**：读一行用 fgets；读了超长输入会缓冲区溢出（越界写坏相邻变量）。
- **2. 数组长度不够放 \0**：char s[5] 存 5 个字符会丢 \0 → 之后 strlen 越界乱读。永远「内容长度 + 1」。
- **3. strcmp 用 == 代替**：a == "Li" 比的是地址，必须 strcmp(a,"Li")==0。
- **4. 中文按字节处理**：UTF-8 下一个汉字 3 字节，strlen("你好")=6 不是 2；%10s 宽度按字节，中文对齐会错位。需要处理中文时按字节流操作，别按「字数」想。

11.7 一分钟自测（先答后揭，答案见练习册）

> **Q1**：字符串 "ab" 在内存里占几个字节?
> **Q2**：scanf("%s", s) 和 fgets(s, sizeof(s), stdin) 的区别?
> **Q3**：strcmp(a, b) 返回 0 表示什么?
> **Q4**：为什么 fgets 读到的字符串常带着一个 \n？怎么去掉?
> **Q5**：strlen("你好") 是多少？为什么?

11.8 练习指引

打开「练习册 · Ch11」：**11-1 填空**热手 → **11-2 改错**（先自己找）→ **11-3/11-4 编程**（fgets + 字符串函数实操）→ **11-5 专业场景**（解析串口 AT 指令，电子信息味）。写完对照参考答案的「常见错法」。

C 语言伴读讲义 · 重点章

Ch12 存储类别、链接和内存管理

对应《C Primer Plus（第6版）》第 12 章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 电子信息方向后续课程与期末常见考点（static/extern、作用域存储期、malloc/free）

12.1 两个问题：变量「管到哪」和「活多久」

每个变量都有两个属性，初学时常被忽略但考试必考：

- **作用域**：在哪些代码里能用到它？——**块作用域**（函数或 {} 内声明，出了块就没了）vs **文件作用域**（函数外面声明，整个文件都能用，也叫全局变量）。
- **存储期**：它的内存什么时候分配、什么时候释放？——**自动存储期**（进入块分配、出块释放，默认的局部变量）vs **静态存储期**（程序启动分配、程序结束才释放）。

```
/* ch12_01_scope.c — 作用域演示 */
#include <stdio.h>

int count = 0;            /* 文件作用域 + 静态存储期（全局）*/

int main(void)
{
    int x = 1;            /* 块作用域 + 自动存储期 */
    {
        int x = 2;        /* 内层块自己的 x，遮蔽外层 */
        printf("内层 x = %d\n", x);
    }
    printf("外层 x = %d\n", x);
    printf("全局 count = %d\n", count);
    return 0;
}
```

12.2 static：让局部变量「记住」上一次的值

函数里普通的局部变量，每次调用都重新分配，但**不会自动初始化**——未赋初值的话，值是栈上残留的不确定值（本机实测同一函数两次调用分别得到 1 和 0）。加 `static` 后，它变成**静态存储期**：只初始化一次（默认置 0），函数退出不销毁，下次调用还留着上次的值。只有 `static` 变量与全局变量默认置 0。

```
/* ch12_02_static.c — static 计数器 */
#include <stdio.h>

int next_id(void)
```

```
{
    static int n = 0;     /* 只初始化一次！ */
    n++;
    return n;
}

int main(void)
{
    printf("%d %d %d\n", next_id(), next_id(), next_id());
    return 0;
}
```

本机 gcc 16.1.0 实测输出：3 2 1。C 标准不规定实参求值顺序（本机是从右向左），所以这种一行多次调用的写法不要用。若去掉 static 且保留初值 0，输出会是 1 1 1（每次调用重新初始化，与「记住上次值」无关）；若连初值都不给，值是栈上残留的不确定值并触发 -Wuninitialized 警告。

> **典型用途**：计数器、只初始化一次的标志位、生成递增 ID（电子信息里给数据帧编号很常见）。

12.3 extern：使用别处定义的全局变量

extern 声明「这个变量在别处定义过，我这里只是引用」。同一个全局变量在多个文件/函数里共享时用它。单文件里，只要变量定义在前面，后面函数直接能用；跨文件时必须在使用方写 extern int count;。

```
/* ch12_03_extern.c — 跨函数共享全局（单文件演示）*/
#include <stdio.h>

int counter = 0;          /* 定义 */

void bump(void)           /* 函数里想用 counter：其实同文件不用 extern */
{
    counter++;
}

int main(void)
{
    bump(); bump(); bump();
    printf("counter = %d\n", counter);
    return 0;
}
```

> **新手坑 1：全局变量不是越多越好。**它让任何函数都能改，调试时不知道谁改的。课程阶段：少用全局，函数间传参优先（这也是 Ch10 讲 const 保护的原因）。

12.4 动态内存：malloc 和 free

前面学的数组长度都是**编译时就定死的**。运行时才知道要多大？用 `malloc` 在**堆**上按需申请：

```
/* ch12_04_malloc.c — 动态数组 */
#include <stdio.h>
#include <stdlib.h>      /* malloc/free 在这里 */

int main(void)
{
    int n, i;
    int *arr;

    printf("要几个数？");
    scanf("%d", &n);
    arr = malloc(n * sizeof(int));          /* 申请 n 个 int；C 里无需 (int*) 强转 */
    if (arr == NULL) {                      /* 申请失败检查！ */
        printf("内存不足\n");
        return 1;
    }
    for (i = 0; i < n; i++) arr[i] = i * i;
    for (i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);                              /* 用完必须还回去 */
    return 0;
}
```

新手坑 2：申请了不 free = 内存泄漏。循环里反复 malloc 不释放，内存越占越多，程序跑久了变慢甚至崩。养成「malloc 和 free 成对写」的习惯。

新手坑 3：free 之后再用。free 只是把内存还给系统，指针还指着那块地址——再去写就是未定义行为。free 后把指针置 NULL 是好习惯。

12.5 常见坑集中营

1. **static 初始化时机**：static 变量初始化只发生一次，且必须是编译期常量（不能用运行时值初始化）。
2. **忘了检查 malloc 返回值**：内存申请失败返回 NULL，直接用就崩。
3. **malloc 和 free 不配对**：申请 3 次只 free 1 次 = 泄漏；free 两次 = 双重释放崩溃。
4. **sizeof 用错**：`malloc(n)` 对 int 数组应该是 `malloc(n * sizeof(int))`，忘了乘 sizeof 会分配太小。

12.6 一分钟自测（答案见练习册）

判断对错：

① 函数内 `static int n = 0;` 每次调用函数时 n 都会被重置成 0。

② `malloc` 申请的内存用完必须 `free`，否则会泄漏。

③ 局部变量的作用域是它所在的整个文件。

12.7 练习指引

做完本章内容后，去练习册完成 Ch12 配套练习；同时打开《C Primer Plus（第6版）》第 12 章，完成**复习题 1、3、5**（题号对应主教材）。本章内容期末常考，static/extern 的理解建议用「把程序运行过程在脑子里演一遍」的方式自测。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译。原书题目请以正版教材为准。